CMSC424: Database Design

Module: Transactions

Instructor: Amol Deshpande
amol@cs.umd.edu

Database Implementation

- Shifting into discussing the internals of a DBMS
  - How data stored? How queries/transactions executed?

- Some of the Topics:
  - Storage: How is data stored? Important features of the storage devices (RAM, Disks, SSDs, etc)
  - Tuple Organization: How are tuples laid out
  - Indexes: How to quickly find specific tuples of interest (e.g., all 'friends' of 'user0')
  - Query processing: How to execute different relational operations? How to combine them to execute an SQL query? How to do this in a parallel setting?
  - Query optimization: How to choose the best way to execute a query?
  - Transactions: How to support the ACID properties? In a distributed setting?

CMSC424: Database Design

Module: Transactions

Transactions Summary

Instructor: Amol Deshpande
amol@umd.edu

Overview

- *Transaction*: A sequence of database actions enclosed within special tags
- Properties:
  - ***Atomicity***: Entire transaction or nothing
  - ***Consistency***: Transaction, executed completely, takes database from one consistent state to another
  - ***Isolation***: Concurrent transactions *appear* to run in isolation
  - ***Durability***: Effects of committed transactions are not lost

- Consistency: Transaction programmer needs to guarantee that
  - DBMS can do a few things, e.g., enforce constraints on the data

- Rest: DBMS guarantees

Examples of Transactions

insert into students values (…)

update instructor
set salary = salary * 1.03

enrolled = select count(*)
from takes
where (course_info) = (CMSC 424, 201, Fall 2022)
if enrolled < capacity for the room:
insert new student into takes for that course

(Modify prerequisites)
delete (CMSC422, CMSC351)
insert (CMSC422, CMSC320)

(Add a new section for a course for a given room and instructor)
if no section currently in that room:
insert a tuple into “sections” with that room
insert a tuple into “teaches”

(Remove a section)
delete from takes for that section
delete from teaches for that section
delete tuple from section

(Switch the advisor for a student)
delete old tuple with that s_id
add new tuple with that s_id and new advisor

Examples of Transactions

```
enrolled = select count(*)
            from takes
            where (course_info) = (CMSC 424, 201, Fall 2022)
if enrolled < capacity for the room:
    insert student A into takes for that course
```

```
enrolled = select count(*)
            from takes
            where (course_info) = (CMSC 424, 201, Fall 2022
if enrolled < capacity for the room:
    insert student B into takes for that course
```

What if two different students tried to enroll at the same time?

Concurrency Control

Examples of Transactions

What if only part of the transaction was completed?

update instructor
set salary = salary * 1.03

(Modify prerequisites)
delete (CMSC422, CMSC351)
insert (CMSC422, CMSC320)

(Remove a section)
delete from takes for that section
delete from teaches for that section
delete tuple from section

(Switch the advisor for a student)
delete old tuple with that s_id
add new tuple with that s_id and new advisor

Recovery

A Schedule

Transactions:

T1: transfers $50 from A to B

T2: transfers 10% of A to B

Database constraint: A + B is constant (*checking+saving accts)*

T1	T2
read(A)	
A = A -50	
write(A)	
read(B)	
B=B+50	
write(B)	
	read(A)
	tmp = A*0.1
	A = A – tmp
	write(A)
	read(B)
	B = B+ tmp
	write(B)

Effect:

	Before	After
A	100	45
B	50	105

Each transaction obeys the constraint.

This schedule does too.

NOTE: This is assuming “global” writes

Another schedule

T1	T2
read(A)	
A = A -50	
write(A)	
	read(A)
	tmp = A*0.1
	A = A – tmp
	write(A)
read(B)	
B=B+50	
write(B)	
	read(B)
	B = B + tmp
	write(B)

Is this schedule okay ?

Lets look at the final effect…

Effect:	Before	After
A	100	45
B	50	105

Consistent.
So this schedule is okay too.

Another schedule

T1	T2
read(A)	
A = A -50	
write(A)	
	read(A)
	tmp = A*0.1
	A = A – tmp
	write(A)
read(B)	
B=B+50	
write(B)	
	read(B)
	B = B+ tmp
	write(B)

Is this schedule okay ?

Lets look at the final effect…

Effect:

	Before	After
A	100	45
B	50	105

Further, the effect same as the serial schedule 1.

Called *serializable*

Example Schedules (Cont.)

A “bad” schedule

T1	T2
read(A)	
A = A -50	
	read(A)
	tmp = A*0.1
	A = A – tmp
	write(A)
	read(B)
write(A)	
read(B)	
B=B+50	
write(B)	
	B = B+ tmp
	write(B)

Effect:	Before	After
A	100	50
B	50	60

Not consistent

Example Schedules – Compute Final Result

Effect:	Before	After
A	100	??
B	50	??

T1	T2	T3
read(A)		
A = A -50		
	read(A)	
	tmp = A*0.1	
	A = A – tmp	
	write(A)	
	read(B)	
		read(A)
		tmp = A*0.2
		A = A + tmp
		write(A)
		read(B)
write(A)		
read(B)		
B=B+50		
write(B)		
	B = B+ tmp	
	write(B)	
		B = B - tmp
		write(B)

T1	T2	T3
read(A)		
A = A -50		
write(A)		
	read(A)	
	tmp = A*0.1	
	A = A – tmp	
	write(A)	
		read(A)
		A = A + 20
		write(A)
read(B)		
B=B+50		
write(B)		
	read(B)	
	B = B+ tmp	
	write(B)	

Find Serializable Schedules

T1	T2	T3	T4	T5
read(A)				
		read(A)		
write(A)				
	read(A)			
		read(B)		
		write(B)		
read(B)				
	write(A)			
				read(C)
	read(B)			
				write(C)
read(C)				
			read(C)	
			write(C)	
	write(B)			

CMSC424: Database Design

Module: Transactions

Concurrency Control: Locking

Instructor: Amol Deshpande
amol@umd.edu

Locking - 1

- Book Chapters
  - 15.1.1-15.1.4, 15.2
- Key topics:
  - Using locking to guarantee concurrency
  - 2-Phase Locking (2PL)
  - Implementation of locking
  - Deadlocks

Lock-based Protocols

- A transaction *must* get a *lock* before operating on the data
- Two types of locks:
  - *Shared* (S) locks (also called *read locks)*
    - Obtained if we want to only read an item – lock-S() instruction
  - *Exclusive* (X) locks (also called *write locks)*
    - Obtained for updating a data item – lock-X() instruction

T1	T2
read(B)	read(A)
B ←B-50	read(B)
write(B)	display(A+B)
read(A)	
A ←A + 50	
write(A)	

T1	T2
lock-X(B)	lock-S(A)
read(B)	read(A)
B ←B-50	unlock(A)
write(B)	lock-S(B)
unlock(B)	read(B)
lock-X(A)	unlock(B)
read(A)	display(A+B)
A ←A + 50	
write(A)	
unlock(A)	

Lock-based Protocols

- Lock requests are made to the *concurrency control manager*
  - It decides whether to *grant* a lock request
- T1 asks for a lock on data item A, and T2 currently has a lock on it ?
  - Depends

T2 lock type	T1 lock type	Should allow ?
Shared	Shared	YES
Shared	Exclusive	NO
Exclusive	-	NO

- If *compatible,* grant the lock, otherwise T1 waits in a *queue.*

Lock-based Protocols

- How do we actually use this to guarantee serializability/recoverability ?
  - Not enough just to take locks when you need to read/write something

T1

lock-X(B)
read(B)
B ←B-50
write(B)
unlock(B)

lock-X(A)
read(A)
A ←A + 50
write(A)
unlock(A)

lock-X(A), lock-X(B)
TMP = (A + B) * 0.1
A = A - TMP
B = B + TMP
unlock(A), unlock(B)

NOT SERIALIZABLE

2-Phase Locking Protocol (2PL)

- Phase 1: Growing phase
  - Transaction may obtain locks
  - But may not release them
- Phase 2: Shrinking phase
  - Transaction may only release locks

- Can be shown that this achieves *serializability*
  - *lock-point*: the time at which a transaction acquired last lock
  - if *lock-point*(T1) < *lock-point*(T2), there is no way for T1 to read any update of T2

T1

lock-X(B)
read(B)
B ← B-50
write(B)
unlock(B)

lock-X(A)
read(A)
A ← A + 50
write(A)
unlock(A)

2 Phase Locking

- Example: T1 in 2PL

	T1
Growing phase	lock-X(B)
	read(B)
	B ← B - 50
	write(B)
	lock-X(A)
	read(A)
	A ← A - 50
	write(A)
Shrinking phase	unlock(B)
	unlock(A)

2 Phase Locking

- Guarantees *serializability*, but not “cascade-less recoverability”

T1	T2	T3
lock-X(A), lock-S(B)		
read(A)		
read(B)		
write(A)		
unlock(A), unlock(B)		
	lock-X(A)	
	read(A)	
	write(A)	
	unlock(A)	
	Commit	lock-S(A)
		read(A)
		Commit
<xction fails>		

2 Phase Locking

- Guarantees *serializability*, but not “cascade-less recoverability”

- Guaranteeing just recoverability:
  - If T2 reads a dirty data of T1 (ie, T1 has not committed), then T2 can’t commit unless T1 either commits or aborts
  - If T1 commits, T2 can proceed with committing
  - If T1 aborts, T2 must abort
    - So cascades still happen

Strict 2PL

- Release *exclusive* locks only at the very end, just after commit or abort

T1	T2	T3
lock-X(A), lock-S(B)		
read(A)		
read(B)		
write(A)		
unlock(A), unlock(B)		
	lock-X(A)	
	read(A)	
	write(A)	
	unlock(A)	
	Commit	
		lock-S(A)
		read(A)
		Commit
<xction fails>		

Strict 2PL will not allow that

Works. Guarantees cascade-less and recoverable schedules.

Strict 2PL - Example

- Why wouldn't this schedule be permitted under Strict 2PL?

T1	**T2**	**T3**	**T4**
	read(A)		
	write(A)		
			read(C)
			write(C)
read(A)			
		read(C)	
	read(C)		
write(A)			
write(B)			
		write(C)	
			read(B)

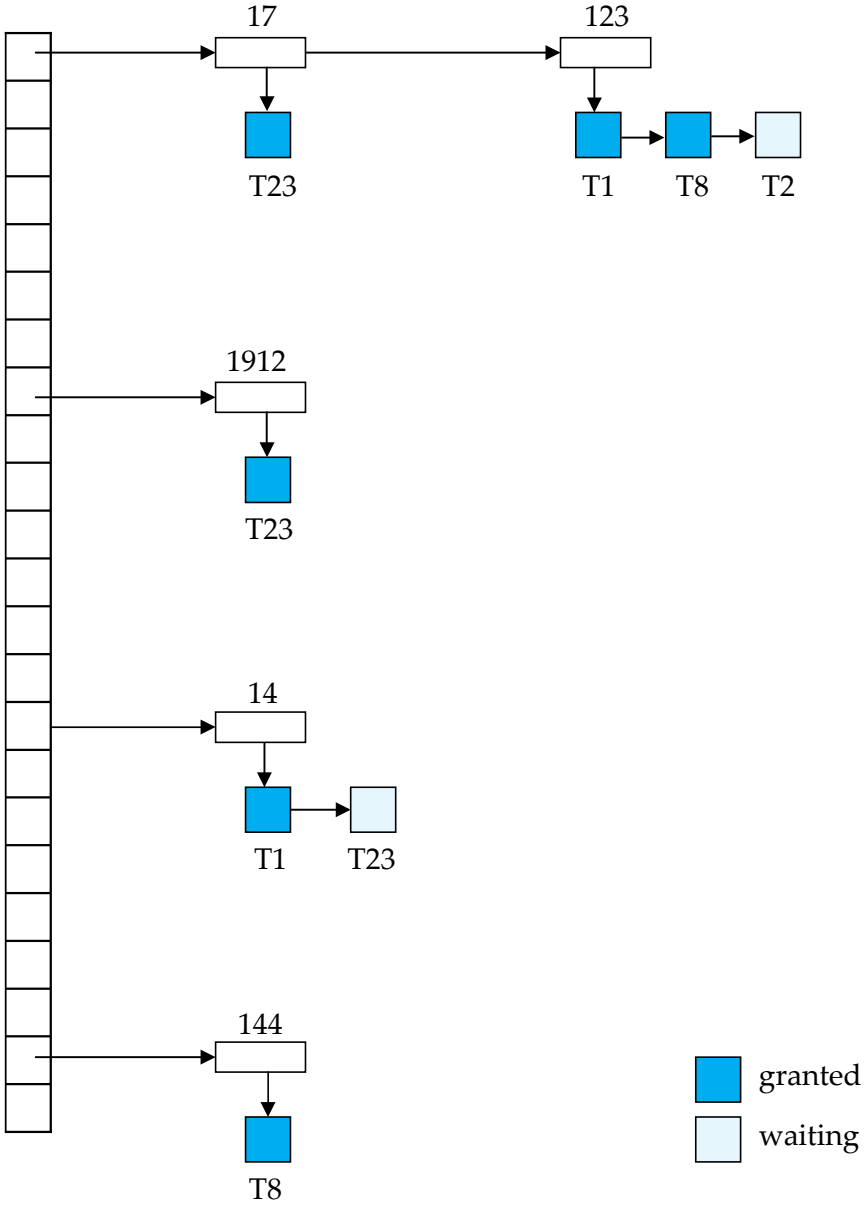
Strict 2PL

- Release *exclusive* locks only at the very end, just before commit or abort
  - Read locks are not important
- Rigorous 2PL: Release both *exclusive and read* locks only at the very end
  - The serializability order === the commit order
  - More intuitive behavior for the users
    - No difference for the system
- Lock conversion:
  - Transaction might not be sure what it needs a write lock on
  - Start with a S lock
  - *Upgrade* to an X lock later if needed
  - Doesn't change any of the other properties of the protocol

Implementation of Locking

- A separate process, or a separate module
- Uses a *lock table* to keep track of currently assigned locks and the requests for locks

Lock Table



- Black rectangles indicate granted locks, white ones indicate waiting requests
- Lock table also records the type of lock granted or requested
- New request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks
- Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted
- If transaction aborts, all waiting or granted requests of the transaction are deleted
  - lock manager may keep a list of locks held by each transaction, to implement this efficiently

More Locking Issues: Deadlocks

- No xction proceeds:

Deadlock

- T1 waits for T2 to unlock A
- T2 waits for T1 to unlock B

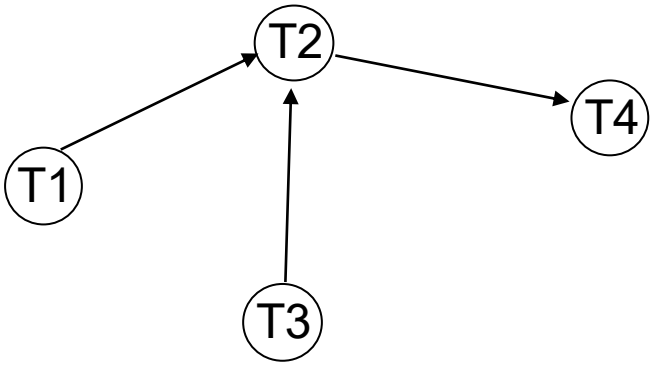
Rollback transactions
Can be costly...

T1	T2
lock-X(B)	
read(B)	
B ← B-50	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

- 2PL does not prevent deadlock
  - Strict doesn't either

Deadlock detection and recovery

- Instead of trying to prevent deadlocks, let them happen and deal with them if they happen
- How do you detect a deadlock?
  - Wait-for graph
  - Directed edge from Ti to Tj
    - Ti waiting for Tj



T1	T2	T3	T4
		X(Z)	
	X(V)		
			X(W)
S(V)			
	S(W)		
		S(V)	

Suppose T4 requests lock-S(Z)....

Dealing with Deadlocks

- Deadlock detected, now what ?
  - Will need to abort some transaction
  - Prefer to abort the one with the minimum work done so far
  - Possibility of starvation
    - If a transaction is aborted too many times, it may be given priority in continueing

Deadlocks -- Example

- Is there a deadlock here? How to resolve it?

Data Item	*Current Lock Holder and Lock Type*	*Transactions Waiting and Lock Type*
A	T4-S, T3-S, T6-S	T2-X
B	T5-X	T1-S
C	T2-S, T3-S, T4-S	T5-X
D	T1-X	T6-S, T4-X

Data Item	Current Lock Holder and Lock Type	Transactions Waiting and Lock Type
A	T1-X	T3-S
B	T4-S, T5-S, T6-S	T1-X
C	T6-S, T5-S, T2-S	T4-X
D	T3-X	T2-S, T6-X

Preventing deadlocks

- Solution 1: A transaction must acquire all locks before it begins
  - Not acceptable in most cases
  - Still need some way to deal with deadlocks during lock acquisition

- Solution 2: A transaction must acquire locks in a particular order over the data items
  - Also called *graph-based protocols*
  - The particular order used doesn't matter (e.g., based on the value of some unique attribute)
  - Guarantees that there can never be a cycle in the precedence graph

Preventing deadlocks

- Solution 3: Use time-stamps; say T1 is older than T2
  - *wait-die scheme:* T1 will wait for T2. T2 will not wait for T1; instead it will abort and restart
    - In the precedence graph, there can be an edge from old transaction to a new transaction, but never the other way
    - So there cannot be a cycle in precedence graph
  - *wound-wait scheme:* T1 will *wound* T2 (force it to abort) if it needs a lock that T2 currently has; T2 will wait for T1.
    - Similar to above: edges only from newer transactions to older transactions
  - May abort more transactions that needed

- Solution 4: Timeout based
  - Transaction waits a certain time for a lock; aborts if it doesn't get it by then
  - As above, may lead to unnecessary restarts, but very simple to implement

CMSC424: Database Design

Module: Transactions

Concurrency Control: Other Schemes

Instructor: Amol Deshpande
amol@umd.edu

1. Time-stamp Based

- Time-stamp based
  - Transactions are issued time-stamps when they enter the system
  - The time-stamps determine the *serializability* order
  - So if T1 entered before T2, then T1 should be before T2 in the serializability order
  - Say *timestamp(T1) < timestamp(T2)*
  - If T1 wants to read data item A
    - If any transaction with larger time-stamp wrote that data item, then this operation is not permitted, and T1 is *aborted*
  - If T1 wants to write data item A
    - If a transaction with larger time-stamp already read that data item or written it, then the write is *rejected* and T1 is aborted
  - Aborted transaction are restarted with a new timestamp
    - Possibility of *starvation*

1. Time-stamp Based

- Maintain for each data Q, two timestamps:
  - W-timestamp(Q): largest time-stamp of any transaction that executed Write(Q) successfully
  - R-timestamp(Q): largest time-stamp of any transaction that executed Read(Q) successfully

- Suppose Ti wants to read(Q):
  - If TS(T_i) < W-Timestamp(Q): Reject the operation and roll back T_i
  - Otherwise, allow the operation and modify:
    - R-timestamp(Q) = max(R-timestamp(Q)), TS(T_i))

1. Time-stamp Based

- Maintain for each data Q, two timestamps:
  - W-timestamp(Q): largest time-stamp of any transaction that executed Write(Q) successfully
  - R-timestamp(Q): largest time-stamp of any transaction that executed Read(Q) successfully

- Suppose Ti wants to write(Q):
  1. If TS(T_i) < R-timestamp(Q): reject the write and roll back T_i
  2. If TS(T_i) < W-timestamp(Q), then T_i is attempting to write an obsolete value of Q.
     - Hence, this **write** operation is rejected, and T_i is rolled back.
  3. Otherwise, execute **write**, and W-timestamp(Q) is set to TS(T_i).

1. Example of Schedule Under TSO

- Is this schedule valid under TSO?

  Assume that initially:
  R-TS(A) = W-TS(A) = 0
  R-TS(B) = W-TS(B) = 0
  Assume TS(T_{25}) = 25 and
  TS(T_{26}) = 26

T_{25}	T_{26}
read(B)	
	read(B)
	$B := B - 50$
	write(B)
read(A)	
	read(A)
display($A + B$)	
	$A := A + 50$
	write(A)
	display($A + B$)

- How about this one, where initially
  R-TS(Q)=W-TS(Q)=0

T_{27}	T_{28}
read(Q)	
	write(Q)
write(Q)	

Example (1)

Consider 5 transactions with timestamp order: T1, T2, T3, T4, T5 (i.e., T1 was the first transaction, etc).

Which transactions would be forced to abort? Answer for each one independently.

Transaction ID	Instructions
T1	read(A), read(C)
T2	read(B)
T1	write(C)
T3	read(A), read(B)
T1	write(A)
T4	read(C), read(D), write(D)
T5	read(B), write(B)
T3	read(D)
T2	read(A), write(A)
T4	read(A)

1. Another Example

- Example

T_1	T_2	T_3	T_4	T_5
				read (X)
	read (Y)			
read (Y)				
		write (Y)		
		write (Z)		
				read (Z)
	read (Z)			
	abort			
read (X)				
			read (W)	
		write (W)		
		abort		
				write (Y)
				write (Z)

2. Optimistic Concurrency Control

- Optimistic concurrency control
  - Also called validation-based
  - Intuition
    - Let the transactions execute as they wish
    - At the very end when they are about to commit, check if there might be any problems/conflicts etc
      - If no, let it commit
      - If yes, abort and restart
  - Optimistic: The hope is that there won't be too many problems/aborts

2. Optimistic Concurrency Control

- Each transaction T_i has 3 timestamps
  - Start(T_i) : the time when T_i started its execution
  - Validation(T_i): the time when T_i entered its validation phase
  - Finish(T_i) : the time when T_i finished its write phase

- Serializability order is determined by timestamp given at validation time, to increase concurrency.
  - Thus TS(T_i) is given the value of Validation(T_i).

- This protocol is useful and gives greater degree of concurrency if probability of conflicts is low.
  - because the serializability order is not pre-decided, and
  - relatively few transactions will have to be rolled back.

2. Optimistic Concurrency Control

- If for all T_i with TS (T_i) < TS (T_j) either one of the following condition holds:
  - **finish**(T_i) < **start**(T_j)
  - **start**(T_j) < **finish**(T_i) < **validation**(T_j) **and** the set of data items written by T_i does not intersect with the set of data items read by T_j.

  then validation succeeds and T_j can be committed. Otherwise, validation fails and T_j is aborted.
- *Justification*: Either the first condition is satisfied, and there is no overlapped execution, or the second condition is satisfied and
  - the writes of T_j do not affect reads of T_i since they occur after T_i has finished its reads.
  - the writes of T_i do not affect reads of T_j since T_j does not read any item written by T_i.

2. Optimistic Concurrency Control

- Example of schedule produced using validation

T_{25}	T_{26}
read (B)	
	read (B)
	$B := B \quad 50$
	read (A)
	$A := A + 50$
read (A)	
⟨ *validate* ⟩	
display ($A + B$)	
	⟨ *validate* ⟩
	write (B)
	write (A)

Example (2)

Explain what checks would be made in each of the validation steps (in order), and whether the validation step succeeds or fails (assume the decisions made for the earlier validate steps, if any, hold).

T1	T2	T3
start, read(A), write(A)		
	start, read(A), read(B), write(B)	
read(B), write(B)		
	validate	
		start, read(A), read(B)
validate		
		validate

Example (3)

Which of the transactions will successfully pass the validation?

Answer for each transaction in isolation -- i.e., assume all other transactions are successful (and finish at the same time as validate) when deciding whether any specific transaction will validate or not.

T1	*T2*	*T3*	*T4*
start, read (A), write(A)			
read(B), write(B)			
	start, read(A)		
validate			
	validate		
		start, read(B), read(C)	
		write(B), write(C)	
			start, read(A)
		validate	
			validate

3. Snapshot Isolation

- Very popular scheme, used as the primary scheme by many systems including Oracle, PostgreSQL etc…
  - Several others support this in addition to locking-based protocol
- A type of “multi-version concurrency control”
  - Also similar to optimistic concurrency control in many ways
- Key idea:
  - For each object, maintain past “versions” of the data along with timestamps
    - Every update to an object causes a new version to be generated

3. Snapshot Isolation

- Read queries:
  - Let “t” be the “time-stamp” of the query, i.e., the time at which it entered the system
  - When the query asks for a data item, provide a version of the data item that was latest as of “t”
    - Even if the data changed in between, provide an old version
  - No locks needed, no waiting for any other transactions or queries
  - The query executes on a consistent snapshot of the database
- Update queries (transactions):
  - Reads processed as above on a snapshot
  - Writes are done in private storage
  - At commit time, for each object that was written, check if some other transaction updated the data item since this transaction started
    - If yes, then abort and restart
    - If no, make all the writes public simultaneously (by making new versions)

3. Snapshot Isolation

- A transaction T1 executing with Snapshot Isolation
  - takes snapshot of committed data at start
  - always reads/modifies data in its own snapshot
  - updates of concurrent transactions are not visible to T1
  - writes of T1 complete when it commits
  - **First-committer-wins rule**:
    - Commits only if no other concurrent transaction has already written data that T1 intends to write.

T1	T2	T3
W(Y := 1) Commit		
	Start R(X) → 0 R(Y)→ 1	
		W(X:=2) W(Z:=3) Commit
	R(Z) → 0 R(Y) → 1 W(X:=3) Commit-Req Abort	

Concurrent updates not visible
Own updates are visible
Not first-committer of X
Serialization error, T2 is rolled back

Example (4)

(1) What values of "A" would T2 and T5 read? See the "??????" for the specific instructions.

(2) Of the three transactions T1, T2, and T4, that have made a Commit-Req, which ones will be allowed to commit?

T1	T2	T3	T4	T5
		Start		
Start R(A) → 1				
	Start R(A) → 1 R(B)→ 2			
			Start R(A) → 1 R(B)→ 2	
		W(A:=2) Commit-Req Commit		
				Start R(A) → ?????
	R(A) → ?????? W(B:=3) Commit-Req			
			Commit-Req	
W(A:=3) Commit-Req				

Example (5)

T2 commits successfully as there is no transaction that has committed since T2 started, and T3 commits successfully as it is a read-only transaction.

- Would T1 be allowed to commit?
- Is this schedule serializable? Clearly explain why or why not.

T1	T2	T3
Start		
Read X = 0		
Read Y = 0		
	Start	
	Read Y = 0	
	Write Y = 20	
	Commit	
		Start
		Read X = 0
		Read Y = 20
		Commit
Set X = X + Y - 11 = -11		
Write X = -11		
Commit-Req		

3. Snapshot Isolation

- Advantages:
  - Read query don't block at all, and run very fast
  - As long as conflicts are rare, update transactions don't abort either
  - Overall better performance than locking-based protocols

- Major disadvantage:
  - Not serializable
  - Inconsistencies may be introduced
  - See the wikipedia article for more details and an example
    - http://en.wikipedia.org/wiki/Snapshot_isolation

3. Snapshot Isolation

- Example of problem with SI
  - T1: x:=y
  - T2: y:= x
  - Initially x = 3 and y = 17
    - Serial execution: x = ??, y = ??
    - if both transactions start at the same time, with snapshot isolation: x = ?? , y = ??
- Called **skew write**
- Skew also occurs with inserts
  - E.g:
    - Find max order number among all orders
    - Create a new order with order number = previous max + 1

3. SI In Oracle and PostgreSQL

- **Warning**: SI used when isolation level is set to serializable, by Oracle, and PostgreSQL versions prior to 9.1
  - PostgreSQL's implementation of SI (versions prior to 9.1) described in Section 26.4.1.3
  - Oracle implements "first updater wins" rule (variant of "first committer wins")
    - concurrent writer check is done at time of write, not at commit time
    - Allows transactions to be rolled back earlier
    - Oracle and PostgreSQL < 9.1 do not support true serializable execution
  - PostgreSQL 9.1 introduced new protocol called "Serializable Snapshot Isolation" (SSI)
    - Which guarantees true serializabilty including handling predicate reads (coming up)

CMSC424: Database Design

Module: Transactions

Recovery: Overview; Terminology; Steal and Force

Instructor: Amol Deshpande
amol@umd.edu

Transactions: Recovery

- Book Chapters
  - 16.1, 16.2, 16.3.2
- Key topics:
  - Challenges in guaranteeing Atomicity and Durability
  - Basics of how disks and memory interact
  - New operations: Output() and Input()
  - STEAL and NO FORCE: Why those are desirable
  - Terminology used in the book: Immediate vs Deferred Modifications

Context

- ACID properties:
  - We have talked about Isolation and Consistency
  - How do we guarantee Atomicity and Durability ?
    - Atomicity: Two problems
      - Part of the transaction is done, but we want to cancel it
        - ABORT/ROLLBACK
      - System crashes during the transaction. Some changes made it to the disk, some didn't.
    - Durability:
      - Transaction finished. User notified. But changes not sent to disk yet (for performance reasons). System crashed.
- Essentially similar solutions

Reasons for crashes

- Transaction failures
  - **Logical errors**: transaction cannot complete due to some internal error condition
  - **System errors**: the database system must terminate an active transaction due to an error condition (e.g., deadlock)
- System crash
  - Power failures, operating system bugs etc
  - **Fail-stop assumption**: non-volatile storage contents are assumed to not be corrupted by system crash
    - Database systems have numerous integrity checks to prevent corruption of disk data
- Disk failure
  - Head crashes; ***for now we will assume***
    - ***STABLE STORAGE:*** ***Data <u>never</u> lost. Can approximate by using RAID and maintaining geographically distant copies of the data***

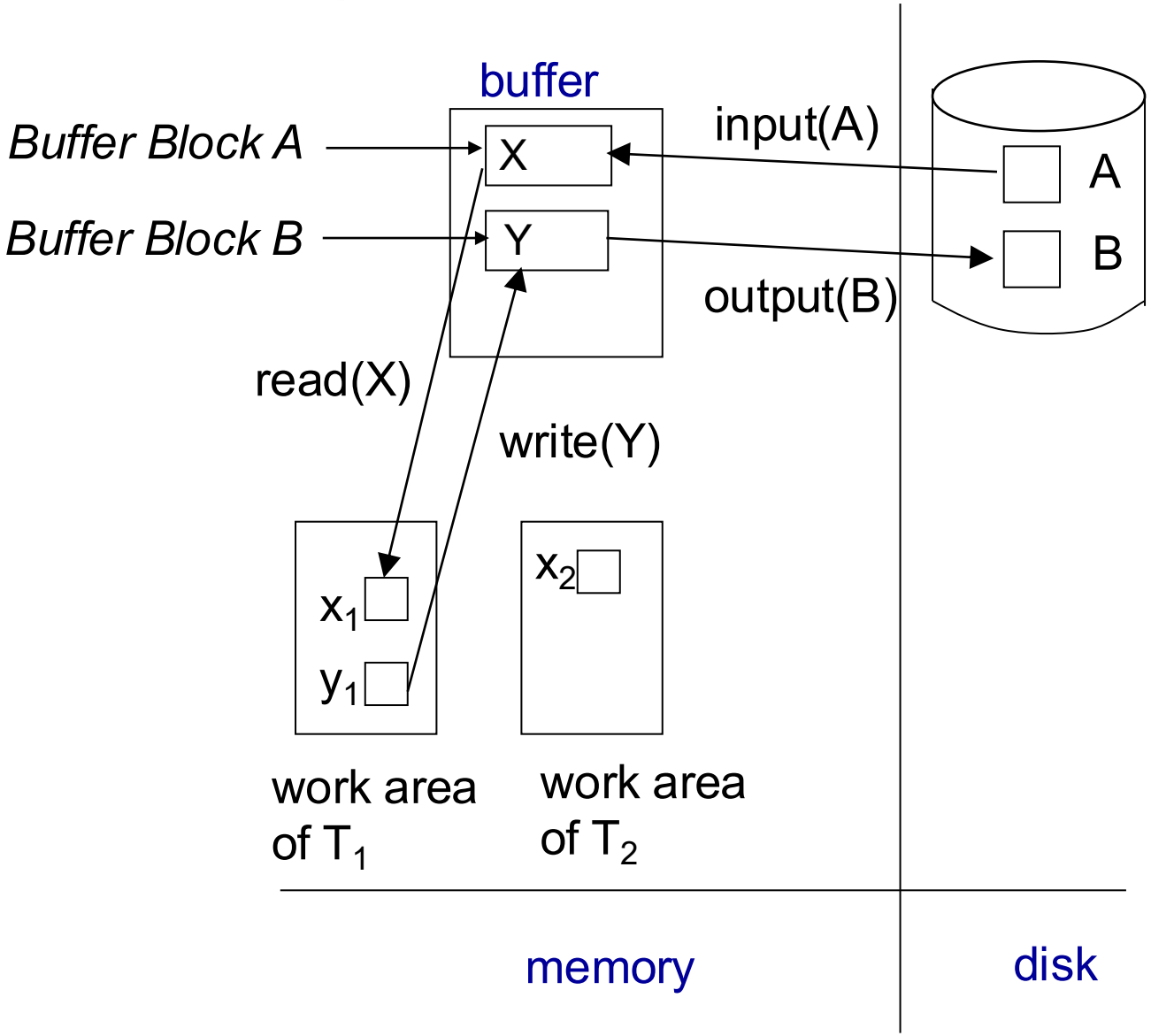
Approach, Assumptions etc..

- Approach:
  - Guarantee A and D:
    - by controlling how the disk and memory interact,
    - by storing enough information during normal processing to recover from failures
    - by developing algorithms to recover the database state
- Assumptions:
  - System may crash, but the *disk is durable*
  - The only *atomicity* guarantee is that *a disk block write* is *atomic*
- Once again, obvious naïve solutions exist that work, but that are too expensive.
  - E.g. The shadow copy solution
    - Make a copy of the database; do the changes on the copy; do an atomic switch of the *dbpointer* at commit time
  - Goal is to do this as efficiently as possible

Data Access

- **Physical blocks** are those blocks residing on the disk.
- **Buffer blocks** are the blocks residing temporarily in main memory.
- Block movements between disk and main memory are initiated through the following two operations:
  - **input**(*B*) transfers the physical block *B* to main memory.
  - **output**(*B*) transfers the buffer block *B* to the disk, and replaces the appropriate physical block there.
- We assume, for simplicity, that each data item fits in, and is stored inside, a single block.

Example of Data Access



Data Access (Cont.)

- Each transaction T_i has its private work-area in which local copies of all data items accessed and updated by it are kept.
  - T_i's local copy of a data item X is called x_i.
- Transferring data items between system buffer blocks and its private work-area done by:
  - **read**(X) assigns the value of data item X to the local variable x_i.
  - **write**(X) assigns the value of local variable x_i to data item {X} in the buffer block.
  - **Note: output**(B_X) need not immediately follow **write**(X). System can perform the **output** operation when it deems fit.
- Transactions
  - Must perform **read**(X) before accessing X for the first time (subsequent reads can be from local copy)
  - **write**(X) can be executed at any time before the transaction commits

STEAL vs NO STEAL, FORCE vs NO FORCE

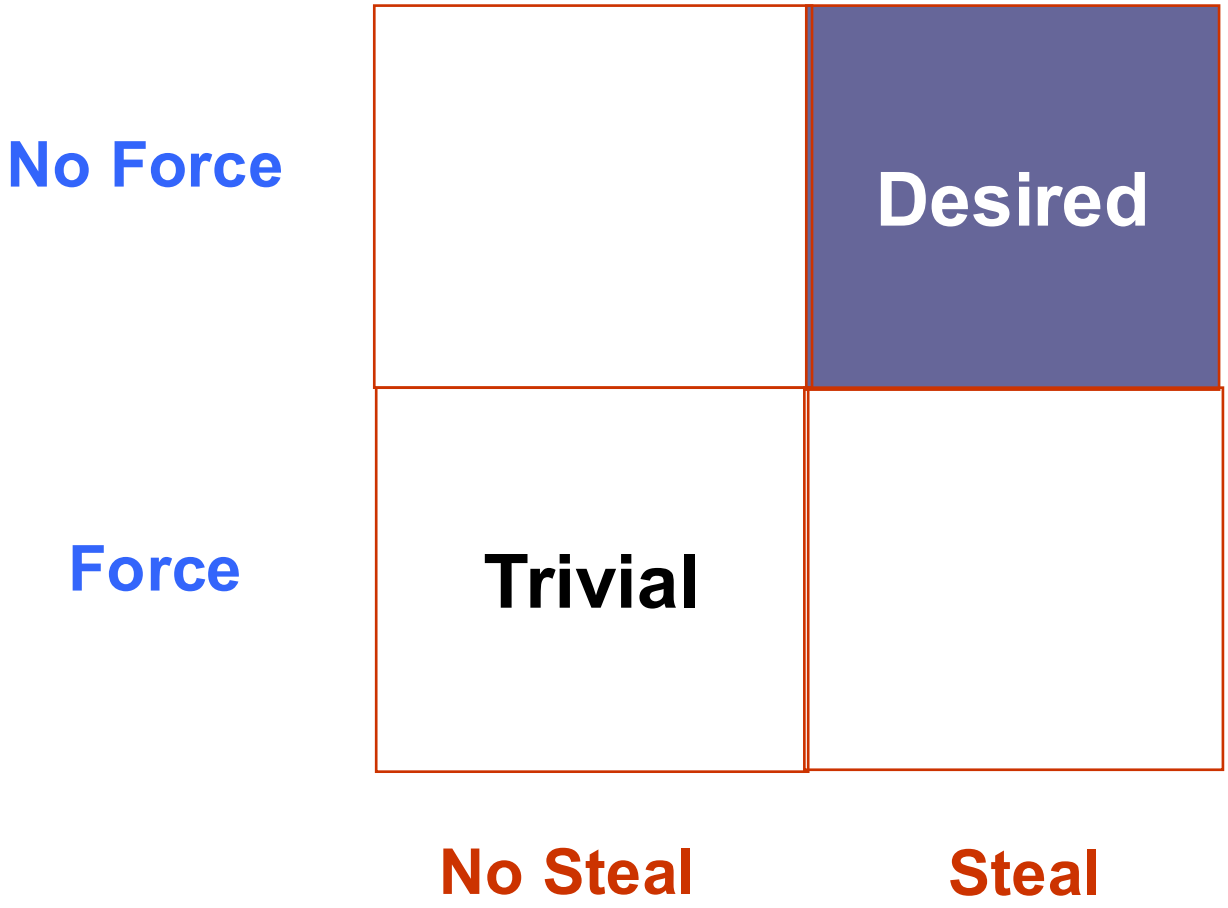
- STEAL:
  - The buffer manager *can steal* a (memory) page from the database
    - ie., it can write an arbitrary page to the disk and use that page for something else from the disk
    - In other words, the database system doesn't control the buffer replacement policy
  - Why a problem ?
    - The page might contain *dirty writes,* ie., writes/updates by a transaction that hasn't committed
  - But, we must allow *steal* for performance reasons.

- NO STEAL:
  - Not allowed. More control, but less flexibility for the buffer manager.

STEAL vs NO STEAL, FORCE vs NO FORCE

- FORCE:
  - The database system *forces* all the updates of a transaction to disk before committing
  - Why ?
    - To make its updates permanent before committing
  - Why a problem ?
    - Most probably random I/Os, so poor response time and throughput
    - Interferes with the disk controlling policies
- NO FORCE:
  - Don’t do the above. Desired.
  - Problem:
    - Guaranteeing durability becomes hard
  - We might still have to *force* some pages to disk, but minimal.

STEAL vs NO STEAL, FORCE vs NO FORCE: Recovery implications



STEAL vs NO STEAL, FORCE vs NO FORCE: Recovery implications

- How to implement A and D when No Steal and Force ?
  - Only updates from committed transaction are written to disk (since no steal)
  - Updates from a transaction are forced to disk before commit (since force)
    - A minor problem: how do you guarantee that all updates from a transaction make it to the disk atomically ?
      - Remember we are only guaranteed an atomic *block write*
      - What if some updates make it to disk, and other don't ?
    - Can use something like shadow copying/shadow paging
  - No atomicity/durability problem arise.

Terminology

- Deferred Database Modification:
  - Similar to NO STEAL, NO FORCE
    - Not identical
  - Only need *redos, no undos*
  - We won't cover this in detail

- Immediate Database Modification:
  - Similar to STEAL, NO FORCE
  - Need both *redos, and undos*

CMSC424: Database Design

Module: Transactions

Recovery: Basics of Logging and UNDO

Instructor: Amol Deshpande
amol@umd.edu

Transactions: Recovery

- Book Chapters
  - 16.3.1, 16.3.5
- Key topics:
  - Generating log records
  - Using log records to support UNDO/Rollback

Log-based Recovery

- Most commonly used recovery method
- Intuitively, a log is a record of everything the database system does
- For every operation done by the database, a *log record* is generated and stored <u>*typically on a different (log) disk*</u>
- *<T1, START>*
- *<T2, COMMIT>*
- *<T2, ABORT>*
- *<T1, A, 100, 200>*
  - T1 modified A; old value = 100, new value = 200

Log

- Example transactions T_0 and T_1 (T_0 executes before T_1):

T_0: **read** (A)
A: - A - 50
write (A)
read (B)
B:- B + 50
write (B)

T_1 : **read** (C)
C:- C- 100
write (C)

- Log:

(a)	(b)	(c)
<T_0 start>	<T_0 start>	<T_0 start>
<T_0, A, 950>	<T_0, A, 950>	<T_0, A, 950>
<T_0, B, 2050>	<T_0, B, 2050>	<T_0, B, 2050>
	<T_0 commit>	<T_0 commit>
	<T_1 start>	<T_1 start>
	<T_1, C, 600>	<T_1, C, 600>
		<T_1 commit>

Log-based Recovery

- Assumptions:
  1. Log records are immediately pushed to the disk as soon as they are generated
  2. Log records are written to disk in the order generated
  3. A log record is generated *before* the actual data value is updated
  4. *Strict two-phase locking*
  - The first assumption can be relaxed
  - As a special case, a transaction is considered *committed* only after the *<T1, COMMIT> has been pushed to the disk*
- But, this seems like exactly what we are trying to avoid ??
  - Log writes are *sequential*
  - They are also typically on a different disk
- Aside: LFS == log-structured file system

Log-based Recovery

- Assumptions:
  1. Log records are immediately pushed to the disk as soon as they are generated
  2. Log records are written to disk in the order generated
  3. A log record is generated *before* the actual data value is updated
  4. *Strict two-phase locking*
  - The first assumption can be relaxed
  - As a special case, a transaction is considered *committed* only after the *<T1, COMMIT> has been pushed to the disk*
- NOTE: As a result of assumptions 1 and 2, if *data item A* is updated, the log record corresponding to the update is always forced to the disk before *data item A* is written to the disk
  - This is actually the only property we need; assumption 1 can be relaxed to just guarantee this (called *write-ahead logging)*

Using the log to *abort/rollback*

- STEAL is allowed, so changes of a transaction may have made it to the disk

- UNDO(T1):
  - Procedure executed to *rollback/undo* the effects of a transaction
  - E.g.
    - *<T1, START>*
    - *<T1, A, 200, 300>*
    - *<T1, B, 400, 300>*
    - *<T1, A, 300, 200>* *[[note: second update of A]]*
    - T1 decides to abort
  - Any of the changes might have made it to the disk

Using the log to *abort/rollback*

- UNDO(T1):
  - Go *backwards* in the *log* looking for log records belonging to T1
  - Restore the values to the old values
  - NOTE: Going backwards is important.
    - *A* was updated twice
  - In the example, we simply:
    - Restore A to 300
    - Restore B to 400
    - Restore A to 200
  - Write a log record <T_i , X_j, V_1>
    - such log records are called **compensation log records**
    - **<T1, A, 300>, <T1, B, 400>, <T1, A, 200>**
  - Note: No other transaction better have changed A or B in the meantime
    - *Strict two-phase locking*

CMSC424: Database Design

Module: Transactions

Recovery: Log-based Restart Recovery

Instructor: Amol Deshpande
amol@umd.edu

Using Logs for Recovery

- Book Chapters
  - 16.4
- Key topics:
  - How to use logs for REDO
  - Idempotency of log records
  - Restart recovery after a failure

Using the log to *recover*

- We don't require FORCE, so a change made by the committed transaction may not have made it to the disk before the system crashed
  - BUT, the log record did (recall our assumptions)
- REDO(T1):
  - Procedure executed to recover a committed transaction
  - E.g.
    - *<T1, START>*
    - *<T1, A, 200, 300>*
    - *<T1, B, 400, 300>*
    - *<T1, A, 300, 200>* *[[note: second update of A]]*
    - *<T1, COMMIT>*
  - By our assumptions, all the log records made it to the disk (since the transaction committed)
  - But any or none of the changes to A or B might have made it to disk

Using the log to *recover*

- REDO(T1):
  - Go *forwards* in the *log* looking for log records belonging to T1
  - Set the values to the new values
  - NOTE: Going forwards is important.
  - In the example, we simply:
    - Set A to 300
    - Set B to 300
    - Set A to 200

Idempotency

- Both redo and undo are required to *idempotent*
  - *F is idempotent, if F(x) = F(F(x)) = F(F(F(F(…F(x)))))*
- Multiple applications shouldn't change the effect
  - This is important because we don't know exactly what made it to the disk, and we can't keep track of that
  - E.g. consider a log record of the type
    - <T1, A, *incremented by 100*>
    - Old value was 200, and so new value was 300
  - But the on disk value might be 200 or 300 (since we have no control over the buffer manager)
  - So we have no idea whether to apply this log record or not
  - Hence, *value based logging* is used (also called *physical)*, not operation based (also called *logical)*

Log-based recovery

- Log is maintained
- If during the normal processing, a transaction needs to abort
  - UNDO() is used for that purpose
- If the system crashes, then we need to do recovery using both UNDO() and REDO()
  - Some transactions that were going on at the time of crash may not have completed, and must be *aborted/undone*
  - Some transaction may have committed, but their changes didn't make it to disk, so they must be *redone*
  - Called *restart recovery*

Recovery Algorithm (Cont.)

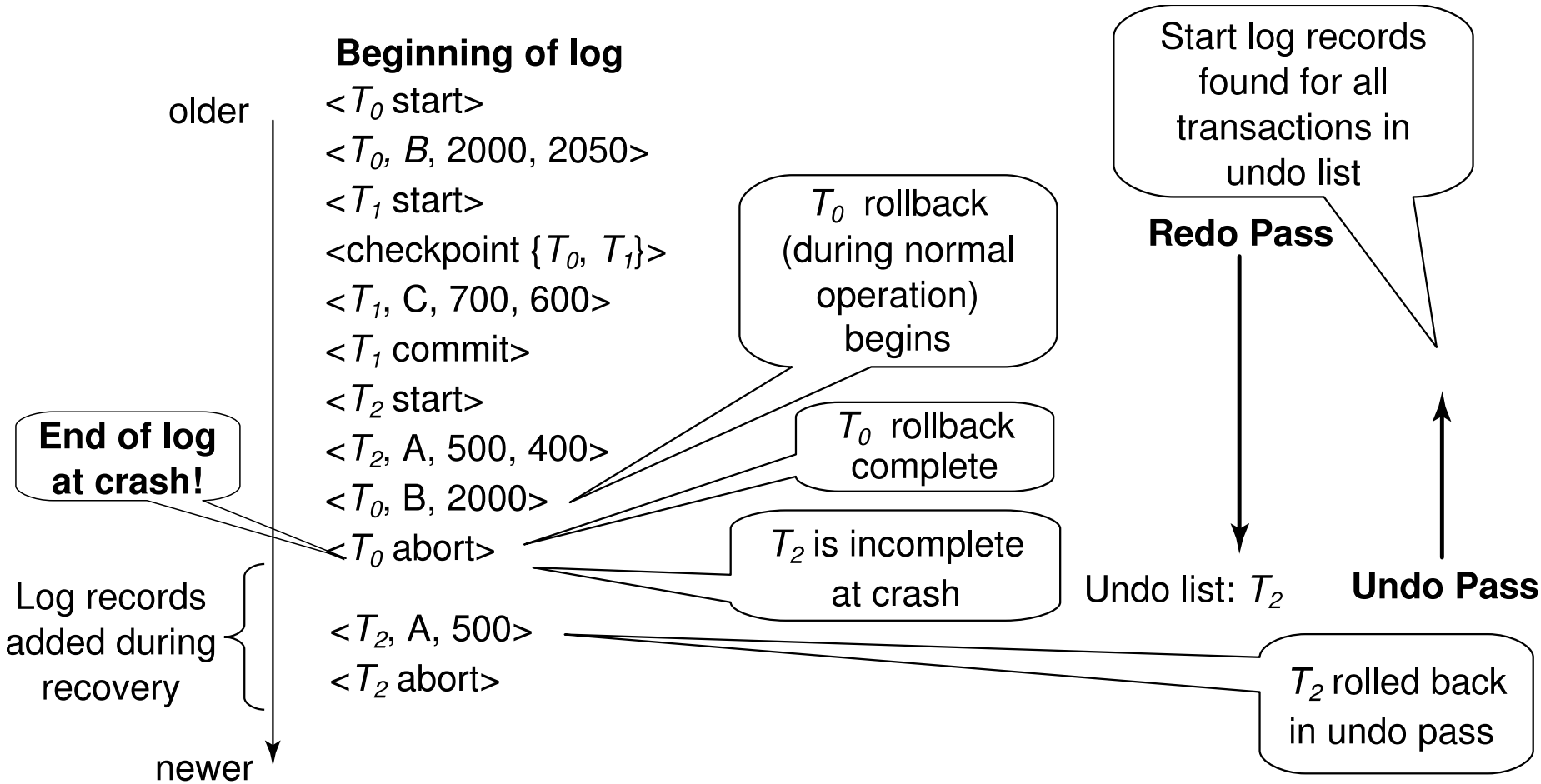
- **Recovery from failure**: Two phases
  - **Redo phase**: replay updates of **all** transactions, whether they committed, aborted, or are incomplete
  - **Undo phase**: undo all incomplete transactions

- **Redo phase**:
  1. Set undo-list to *{} (empty)*.
  2. Scan forward from first log record
     1. Whenever a record <T_i, X_j, V_1, V_2> is found, redo it by writing V_2 to X_j
     2. Whenever a log record <T_i **start**> is found, add T_i to undo-list
     3. Whenever a log record <T_i **commit**> *or* <T_i **abort**> is found, remove T_i from undo-list

Recovery Algorithm (Cont.)

- **Undo phase:**
  1. Scan log backwards from end
     1. Whenever a log record <T_i, X_j, V_1, V_2> is found where T_i is in undo-list perform same actions as for transaction rollback:
        1. perform undo by writing V_1 to X_j.
        2. write a log record <T_i , X_j, V_1>
     2. Whenever a log record <T_i **start**> is found where T_i is in undo-list,
        1. Write a log record <T_i **abort**>
        2. Remove T_i from undo-list
     3. Stop when undo-list is empty
        - i.e. <T_i **start**> has been found for every transaction in undo-list
- After undo phase completes, normal transaction processing can commence

Example of Recovery



CMSC424: Database Design

Module: Transactions

Checkpointing; Write-ahead Logging; Recap

Instructor: Amol Deshpande
amol@umd.edu

Recovery: Recap

- Book Chapters
  - 16.3.6, 16.5
- Key topics:
  - Checkpointing
  - Write-ahead logging
  - Recap

Checkpointing

- How far should we go back in the log while constructing redo and undo lists ??
  - It is possible that a transaction made an update at the very beginning of the system, and that update never made it to disk
    - very very unlikely, but possible (because we don't do force)
  - For correctness, we have to go back all the way to the beginning of the log
  - Bad idea !!

- Checkpointing is a mechanism to reduce this

Checkpointing

- Periodically, the database system writes out everything in the memory to disk
  - Goal is to get the database in a state that we know (not necessarily consistent state)
- Steps:
  - Stop all other activity in the database system
  - Write out the entire contents of the memory to the disk
    - Only need to write updated pages, so not so bad
    - Entire === all updates, whether committed or not
  - Write out all the log records to the disk
  - Write out a special log record to disk
    - *<CHECKPOINT LIST_OF_ACTIVE_TRANSACTIONS>*
    - The second component is the list of all active transactions in the system right now
  - Continue with the transactions again

Recovery Algorithm (Cont.)

- **Recovery from failure**: Two phases
  - **Redo phase**: replay updates of **all** transactions, whether they committed, aborted, or are incomplete
  - **Undo phase**: undo all incomplete transactions

- **Redo phase** (No difference for Undo phase):
  1. Find last <**checkpoint** *L*> record, and set undo-list to *L*.

     - If no checkpoint record, start at the beginning
  2. Scan forward from above <**checkpoint** *L*> record
     1. Whenever a record <T_i, X_j, V_1, V_2> is found, redo it by writing V_2 to X_j
     2. Whenever a log record <T_i **start**> is found, add T_i to undo-list
     3. Whenever a log record <T_i **commit**> *or* <T_i **abort**> is found, remove T_i from undo-list

Recap so far …

- Log-based recovery
  - Uses a *log* to aid during recovery
- UNDO()
  - Used for normal transaction abort/rollback, as well as during restart recovery
- REDO()
  - Used during restart recovery
- Checkpoints
  - Used to reduce the restart recovery time

Write-ahead logging

- We assumed that log records are written to disk as soon as generated
  - Too restrictive
- Write-ahead logging:
  - Before an update on a data item (say A) makes it to disk, the log records referring to the update must be forced to disk
  - How ?
    - Each log record has a log sequence number (LSN)
      - Monotonically increasing
    - For each page in the memory, we maintain the LSN of the *<u>last log record</u>* that updated a record on this page
      - *pageLSN*
    - If a page *P* is to be written to disk, all the log records till *pageLSN(P)* are forced to disk

Write-ahead logging

- Write-ahead logging (WAL) is sufficient for all our purposes
  - All the algorithms discussed before work

- Note the special case:
  - A transaction is not considered committed, unless the <T, commit> record is on disk

Other issues

- The system halts during checkpointing
  - Not acceptable
  - Advanced recovery techniques allow the system to continue processing while checkpointing is going on
- System may crash during recovery
  - Our simple protocol is actually fine
  - In general, this can be painful to handle
- B+-Tree and other indexing techniques
  - Strict 2PL is typically not followed (we didn't cover this)
  - So physical logging is not sufficient; must have logical logging

Other issues

- ARIES: Considered *the canonical description of log-based recovery*
  - Used in most systems
  - Has many other types of log records that simplify recovery significantly

- Loss of disk:
  - Can use a scheme similar to checkpoining to periodically dump the database onto *tapes* or *optical storage*
  - Techniques exist for doing this while the transactions are executing (called *fuzzy dumps)*

- Shadow paging:
  - Read up

Recap

- STEAL vs NO STEAL, FORCE vs NO FORCE
  - We studied how to do STEAL and NO FORCE through log-based recovery scheme

	No Steal	Steal
No Force		Desired
Force	Trivial	

	No Steal	Steal
No Force	REDO NO UNDO	REDO UNDO
Force	NO REDO NO UNDO	NO REDO UNDO

Recap

- ACID Properties
  - Atomicity and Durability :
    - Logs, undo(), redo(), WAL etc
  - Consistency and Isolation:
    - Concurrency schemes
  - Strong interactions:
    - We had to assume Strict 2PL for proving correctness of recovery

CMSC424: Database Design

Module: Transactions

Distributed Transactions

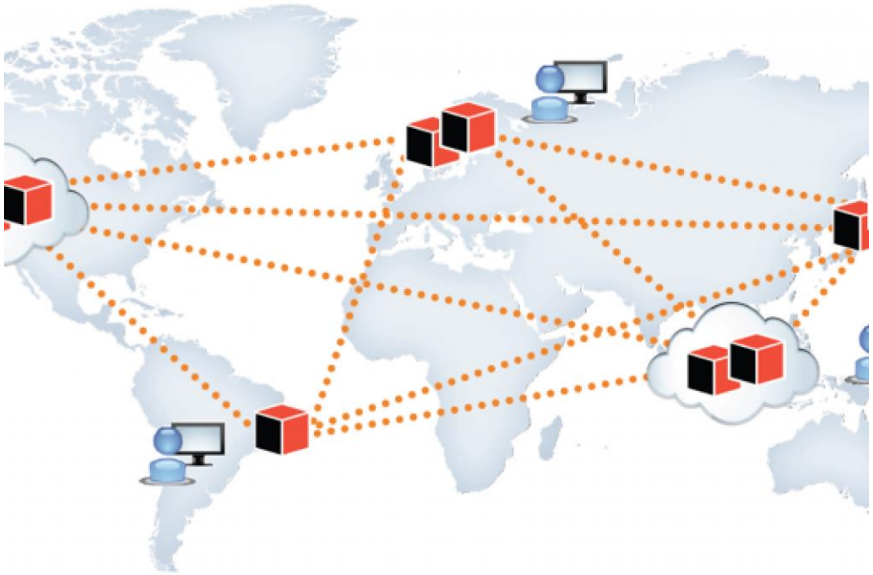
Instructor: Amol Deshpande
amol@umd.edu

Distributed Transactions

- Book Chapters
  - 19.1-19.4, 19.6: at a fairly high level
- Key topics:
  - Distributed databases and replication
  - Transaction processing in distributed databases
  - 2-Phase Commit
  - Brief discussion of other protocols including Paxos

Distributed Database System

- A distributed database system consists of loosely coupled sites that share no physical component
- Database systems that run on each site are independent of each other
  - Or not – lot of variations here
- Transactions may access data at one or more sites
  - Because of replication, even updating a single data item involves a "distributed transaction" (to keep all replicas up to date)

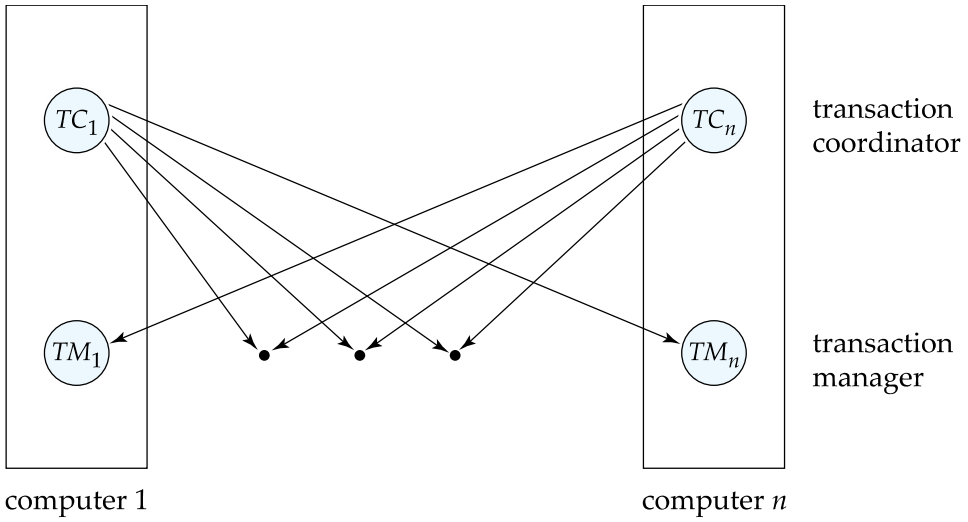


Data Replication

- A relation or fragment of a relation is **replicated** if it is stored redundantly in two or more sites
- Advantages:
  - **Availability**: failures can be handled through replicas
  - **Parallelism**: queries can be run on any replica
  - **Reduced data transfer**: queries can go to the "closest" replica
- Disadvantages:
  - **Increased cost of updates:** both computation as well as latency
  - **Increased complexity of concurrency control:** need to update all copies of a data item/tuple
- **Typically we use the term "data items", which may be tuples or relations or relation partitions**

Distributed Transactions

- Transaction may access data at several sites
  - As noted, single data item update is also a distributed transaction
- **Each site has a local transaction manager** responsible for:
  - Maintaining a log for recovery purposes
  - Coordinating the concurrent execution of the transactions
- **Each site has a transaction coordinator,** which is responsible for:
  - Starting the execution of transactions that originate at the site.
  - Distributing sub-transactions at appropriate sites for execution.
  - Coordinating the termination of each transaction that originates at the site -- transaction may commit at all sites or abort at all sites.



System Failure Modes

- Failures unique to distributed systems:
  - Failure of a site.
  - Loss of massages
    - Handled by network transmission control protocols such as TCP-IP
  - Failure of a communication link
    - Handled by network protocols, by routing messages via alternative links
  - **Network partition**
    - A network is said to be **partitioned** when it has been split into two or more subsystems that lack any connection between them
      - Note: a subsystem may consist of a single node
- Network partitioning and site failures are generally indistinguishable.

Commit Protocols

- Commit protocols are used to ensure atomicity across sites
  - a transaction which executes at multiple sites must either be committed at all the sites, or aborted at all the sites.
  - not acceptable to have a transaction committed at one site and aborted at another
- **Two-phase *commit* (2PC)** protocol is widely used
- **Three-phase commit (3PC)** protocol
  - Handles some situations that 2PC doesn't
  - Not widely used
- **Paxos**
  - Robust alternative to 2PC that handles more situations as well
  - Was considered too expensive at one point, but widely used today
- **RAFT**: Alternative to Paxos

Two Phase Commit Protocol (2PC)

- Assumes **fail-stop** model – failed sites simply stop working, and do not cause any other harm, such as sending incorrect messages to other sites.
- Execution of the protocol is initiated by the coordinator after the last step of the transaction has been reached.
- The protocol involves all the local sites at which the transaction executed
- Let T be a transaction initiated at site S_i, and let the transaction coordinator at S_i be C_i

Two Phase Commit Protocol (2PC)

Coordinator Log	Messages	Subordinate Log
	PREPARE $\rightarrow$	
		prepare*/abort*
	$\leftarrow$ VOTE YES/NO	
commit*/abort*		
	COMMIT/ABORT$\rightarrow$	
		commit*/abort*
	$\leftarrow$ ACK	
end		

Goal: Make sure all ”sites” commit or abort

Assumption: Some log records can be “forced” (denote * above)

Phase 1: Obtaining a Decision

- Coordinator asks all participants to *prepare* to commit transaction T_i.
  - C_i adds the records <**prepare** *T*> to the log and forces log to stable storage
  - sends **prepare** *T* messages to all sites at which *T* executed

- Upon receiving message, transaction manager at site determines if it can commit the transaction
  - if not, add a record <**no** *T*> to the log and send **abort** *T* message to C_i
  - if the transaction can be committed, then:
  - add the record <**ready** *T*> to the log
  - force *all records* for *T* to stable storage
  - send **ready** *T* message to C_i

Phase 2: Recording the Decision

- *T* can be committed of C_i received a **ready** *T* message from all the participating sites: otherwise *T* must be aborted.
- Coordinator adds a decision record, <**commit** *T*> or <a**bort** *T*>, to the log and forces record onto stable storage. Once the record stable storage it is irrevocable (even if failures occur)
- Coordinator sends a message to each participant informing it of the decision (commit or abort)
- Participants take appropriate action locally.

Handling of Failures - Site Failure

When site S_i recovers, it examines its log to determine the fate of transactions active at the time of the failure.

- Log contain <**commit** *T*> record: txn had completed, nothing to be done
- Log contains <**abort** *T*> record: txn had completed, nothing to be done
- Log contains <**ready** *T*> record: site must consult C_i to determine the fate of *T*.
  - If *T* committed, **redo** (*T*); write <**commit** *T*> record
  - If *T* aborted, **undo** (*T*)
- The log contains no log records concerning *T*:
  - Implies that S_k failed before responding to the **prepare** *T* message from C_i
  - since the failure of S_k precludes the sending of such a response, coordinator C_1 must abort *T*
  - S_k must execute **undo** (*T*)

Handling of Failures- Coordinator Failure

- If coordinator fails while the commit protocol for *T* is executing then participating sites must decide on *T*' s fate:
  1. If an active site contains a <**commit** *T*> record in its log, then *T* must be committed.
  2. If an active site contains an <**abort** *T*> record in its log, then *T* must be aborted.
  3. If some active participating site does not contain a <**ready** *T*> record in its log, then the failed coordinator C_i cannot have decided to commit *T*.
     - Can therefore abort *T;* however, such a site must reject any subsequent <**prepare** *T*> message from C_i
  4. If none of the above cases holds, then all active sites must have a <**ready** *T*> record in their logs, but no additional control records (such as <**abort** *T*> of <**commit** *T*>).
     - In this case active sites must wait for C_i to recover, to find decision.
- **Blocking problem**: active sites may have to wait for failed coordinator to recover.

Handling of Failures - Network Partition

- If the coordinator and all its participants remain in one partition, the failure has no effect on the commit protocol.
- If the coordinator and its participants belong to several partitions:
  - Sites that are not in the partition containing the coordinator think the coordinator has failed, and execute the protocol to deal with failure of the coordinator.
    - No harm results, but sites may still have to wait for decision from coordinator.
- The coordinator and the sites are in the same partition as the coordinator think that the sites in the other partition have failed, and follow the usual commit protocol.
    - Again, no harm results

More…

- Three-phase Commit
  - 2PC can't handle failure of a coordinator well – everything halts waiting for the coordinator to come back up
  - Three-phase commit handles that through another phase

- Paxos and RAFT
  - Solutions for the "consensus problem": get a collection of distributed entities to "choose" a single value
    - In case of transaction, you are choosing abort/commit
  - Fairly complex, but well-understood today
  - Widely used in most distributed systems today
  - See the Wikipedia pages
  - A nice recent paper: **Paxos vs Raft: Have we reached consensus on distributed consensus? – Heidi Howard, 2020**

More…

- Bitcoin (and other cryptocurrencies)
  - Fundamental problem is the same one, of obtaining “consensus”
    - But need to support a large number of entities, 1000s or more
    - Can’t assume full one-to-one communication
  - Instead:
    - Choose a “leader” based on ”proof of work”
      - Whoever solves a hard puzzle first becomes the “leader”
    - The ”leader” chooses the next “block” in the blockchain
      - A block is basically a list of transactions to accept
    - Reward the puzzle solvers with money (“bitcoins”)
      - So they have an incentive to keep solving puzzles
  - Blockchain?
    - Blockchain is a small part of bitcoin
    - A cryptographically designed chain of blocks that are immutable